CSCI-360 Machine Learning Final Competition 26s
CSI500 Stock Selection Report

Mingqian Yang

11 May 2026

Path note. All file paths and commands referenced in this report are relative to the cleaned project repository available at https://github.com/TimmoTaffy/ml-competition-sp26_gradescope. This note matters because an earlier Phase2 code ZIP was submitted before the documentation cleanup and reproducibility edits, and the current Gradescope report channel does not accept a code ZIP upload. The earlier ZIP does not prevent reproduction; the GitHub repository above contains the final cleaned code, configuration files, report source, and reproducibility records.

Executive Summary

The final system trains XGBoost regressors from scratch to rank CSI500 stocks by expected five-trading-day excess return over the CSI500 index. The Phase1 submission used the strongest single candidate, `stable_compact_current`. After the Phase1 window became public, Phase2 used a more robust portfolio-level blend of `stable_compact_current` and `stable_compact_capacity`: 0.45 current plus 0.55 capacity, truncated to 44 holdings.

The workflow does not reserve a separate static test set for final model selection. This is intentional: the task is a live portfolio competition with very few recent trading days, and the final judge is the next unseen holding window. Instead, the workflow uses strict time-causal live-style walk-forward validation for model and portfolio selection, then reports the two course live windows as external out-of-sample checks. The Phase1 submitted portfolio earned +4.834% excess return from May 6–8, compared with +2.693% for the instructor XGBoost baseline regenerated under the same as-of convention.

1 Factors

The feature design started from the view that most single short-horizon market features are noisy. Instead of relying on one hand-coded rule, the project constructed a large but economically grouped feature pool and selected features by rolling validation.

The report lists factor groups and representative examples rather than every raw column. The complete reproducible feature universe is in `data/final_feature_columns.txt`; the exact retained lists are in `experiments/research_families/fine_tune_top12/feature_sets/`; and the feature engineering notes are in `docs/FEATURES_AND_SELECTION.md`.

Group	Examples	Rationale
Technical and momentum	1/2/3/5/10/20/60/120-day returns, volatility, RSI, moving-average distance	Captures short-horizon continuation and reversal patterns.
CSI500-relative	excess returns versus CSI500, beta, residual momentum, relative volatility	Aligns the prediction target with the live excess-return objective.
Bollinger bands	band z-score, position, width, lower/upper proximity, near/breakout flags	Tests mean-reversion and breakout effects without hard-coding direction.
Liquidity and risk	halt flags, zero-volume days, missing days, turnover, ATR, drawdown	Avoids wasting weight on untradeable or high-tail-risk names.
Industry-relative	Shenwan industry returns and stock-minus-industry ranks	Separates stock selection from sector rotation.
Valuation and quality	PE/PB, book-to-price, ROE, ROA, margins, 3-year revenue/profit growth, cash-flow quality	Adds slower-moving economic information beyond price noise.
Market regime	broad indices, ETFs, sector ETF proxies, crude/gold/global proxies, CSI500 quality-growth index	Allows the model to react to style and macro regime shifts.

AH-premium features were removed because CSI500 dual-listed coverage was too low for a stable cross-sectional signal. Feature selection was profile-specific: the general model and short-history model used separate selected lists. The candidate ranking families `balanced`, `stable`, `recent`, `economic`, and `short_aggressive` changed the feature-selection objective, then all candidates were compared by the same portfolio validation standard.

1.1 Feature Selection Process

The selection pipeline was designed to avoid trusting one model's feature importance. For each profile, features were first tested as single factors over rolling historical windows. The single-factor checks included rank IC, top decile forward excess return, top-minus-bottom spread, hit rate, worst-window behavior, missingness, and cross-window stability.

Next, grouped model tests checked whether economically coherent feature groups improved a portfolio-like objective. This matters because a feature with low standalone IC can still help in combination, while a noisy feature can look good in one window by chance. The combined ranking

score rewarded stable positive rank IC, positive top-decile excess, good downside behavior, and useful group ablation results, while penalizing unstable or sparse signals.

Ranking family	Reasoning
`balanced`	Broad default score across IC, top-decile return, and stability.
`stable`	Emphasizes lower missingness, downside control, and stable signs.
`recent`	Gives more weight to recent windows, useful for regime shifts.
`economic`	Gives more weight to quality, valuation, and fundamental groups.
`short_aggressive`	Emphasizes short-horizon technical and recent signals.

The reason for keeping multiple ranking families is that the feature-ranking step interacts with model capacity and portfolio construction. Instead of declaring one feature ranking "correct", I let several economically defensible rankings compete under the same live-style portfolio validation.

Concrete feature-selection inputs are in `configs/feature_selection_general.json` and `configs/feature_selection_short_history.json`. The ranking-family logic is implemented in `tune_research_families` The selected feature lists for retained candidates are under `experiments/.../feature_sets/`; the complete path map is in `docs/MODEL_SELECTION_LINEAGE.md`.

2 Models

Each source model is an `XGBRegressor` trained from scratch. A training row is one stock on one date, and the model predicts `target_excess_5d`. The final structure has two profiles:

Profile	Role
`general`	Broader cross-sectional signal with more history and more features.
`short_history`	Recent technical/regime signal with shorter training lookback.

Predictions are converted to cross-sectional percentile ranks before blending:

$$\text{combined score} = w_g \cdot \text{rank}(\hat{y}_g) + w_s \cdot \text{rank}(\hat{y}_s).$$

Using ranks avoids mixing incompatible raw XGBoost score scales. The portfolio builder sorts by combined score, applies the risk filter, selects `top_k`, assigns equal or rank weights, caps concentration, and renormalizes to one.

The general/short split was kept because the two profiles answer different questions. The general model uses broader history and is better suited for stable cross-sectional relationships. The short-history model uses a shorter lookback and is better suited for recent technical or regime effects. Their raw predictions are not directly comparable, so the ensemble uses percentile ranks instead of raw scores. The weight between them is not fixed by hand; it is a portfolio hyperparameter selected during validation.

The default profile configs are `configs/model_general.json` and `configs/model_short_history.json`. The final current variants are `general_top80_shallow` and `short_history_top60_recent_train`; the final capacity variants are `general_top80_base` and `short_history_top20_deep_interaction`. Exact JSON paths and selected feature-list paths are listed in `docs/HYPERPARAMETERS.md` and `docs/PHASE_REPRODUCTION.md`. Final ensemble/portfolio settings are in `configs/ensemble_phase1.json`, `configs/ensemble_stable_compact_capacity.json`, and the Phase2 blend config in `experiments/portfolio_blend_current_capacity/`.

2.1 Hyperparameter And Candidate Search

Hyperparameters were selected through a staged workflow:

1. Build the feature matrix from public data with announcement-date-aware fundamentals.
2. Run rolling feature selection for general and short-history profiles.
3. Search structured research families: ranking family, feature-count pair, and XGB preset.
4. Retain a frozen top12 candidate set.
5. Re-evaluate retained candidates with strict live-style walk-forward validation.
6. Fine-tune only portfolio construction on 3/7/13 recent non-overlapping 5-day windows.
7. For Phase2, tune a portfolio-level current/capacity blend over alpha and `top_k`.

The structured research search crossed:

5 feature-ranking families × 4 feature-count pairs × 3 XGBoost presets = 60 candidate families.

The feature-count pairs were `compact`, `medium`, `wide`, and `extra_wide`. The XGBoost presets were `conservative`, `current`, and `capacity`. `conservative` used shallower/more regularized trees; `current` was close to a reasonable baseline tree shape; and `capacity` allowed more interactions. The search first narrowed 60 candidates to a smaller pool, then froze 12 candidates for strict live-style validation and portfolio tuning.

The 60-candidate construction is documented in `docs/MODEL_SELECTION_LINEAGE.md` and implemented in `tune_research_families.py`. The retained top33/top12 audit trail is in `candidate_family_lookup.csv` under the fine-tune experiment directory; the frozen top12 candidate directories are under `experiments/.../candidates/`. The readable top12 rerun guide is `docs/TOP12_REPRODUCTION.md`.

The final candidate comparison did not optimize standalone IC. It optimized a portfolio-oriented validation score. For each historical window, the system retrained from scratch, created a legal portfolio, and measured realized excess return. Candidate scoring combined mean excess, worst-window excess, and hit rate, because the live competition rewards realized portfolio excess rather than prediction correlation alone.

Live-style validation was also used to compare the final Phase2 source candidates. For each historical 5-day window, the system retrained from scratch at the entry date using only labels known then, built a legal portfolio, and scored the following window. On the latest 13-window run ending 2026-05-08:

Candidate	Mean excess	Hit rate	Worst window	Live score
`stable_compact_capacity`	+0.881%	76.9%	−2.265%	0.795
`stable_compact_current`	+1.211%	61.5%	−3.734%	0.711

The current candidate had higher mean excess but worse drawdown stability. The capacity candidate had lower mean but better hit rate and worst-window control. This tradeoff motivated Phase2's portfolio-level blend. Complete top12 rankings, including candidates not used in the final Phase2 blend, are recorded in `docs/TOP12_REPRODUCTION.md` and `experiments/.../live_portfolio_fine_tune_3_7_13/leaderboard.csv`.

Item	Phase1	Phase2
Source candidate(s)	`stable_compact_current`	`stable_compact_current`, `stable_compact_capacity`
Current variants	`general_top80_shallow`, `short_history_top60_recent_train`	same
Capacity variants	not used	`general_top80_base`, `short_history_top20_deep_interaction`
Current score weight	0.55 / 0.45	inside source portfolio
Capacity score weight	not used	0.978 / 0.022
Final portfolio	31-name rank-weighted	0.45 current + 0.55 capacity, top 44
Risk filter	enabled	enabled in both source portfolios

2.2 Phase1 Selection

Phase1 was selected before the first live window. The strongest retained candidate at that point was `stable_compact_current`. The design process was:

1. Generate 60 structured candidates from five feature-ranking families, four feature-count pairs, and three XGBoost presets.
2. Use the early broad/fine research stage to reduce the search space to a top33 pool, then freeze 12 candidate families for final audit.
3. Re-evaluate those 12 under live-style walk-forward validation.
4. Fine-tune only portfolio construction on the retained candidates: general/short rank weight, `top_k`, internal cap, risk filter, and equal/rank weighting.
5. Run a robustness audit around the best rounded parameters.

The final Phase1 choice was `stable_compact_current`. It combined `general_top80_shallow` and `short_history_top60_recent_train`. The fine-tuned winner used approximately 0.55/0.45 general/short rank blending and a 31-stock rank-weighted portfolio. The exact validation winner had general_weight = 0.5465 and internal_cap = 0.031758; these were rounded to 0.55 and 0.032 because the robustness audit showed negligible validation score loss.

2.3 Phase2 Selection

Phase2 was selected after the May 6–8 Phase1 window became public. At that point, `stable_compact_current` still had high mean excess but also showed larger worst-window risk and parameter sensitivity. `stable_compact_capacity` had lower mean excess but better robustness in the latest live-style validation.

The key design decision for Phase2 was to blend portfolios, not raw model scores. This preserves each source candidate's separately tuned legal portfolio construction and avoids assuming that raw XGBoost scores from different candidates are on the same scale. The Phase2 tuner compared pure current, pure capacity, and portfolio blends on the same 3/7/13 live-style validation anchors.

The final grid searched current weight α and final `top_k`. Both the active robust score and the older 3/7/13 score selected `final_portfolio_a0.45_k44`, i.e. 0.45 current plus 0.55 capacity with 44 final holdings. This choice kept part of the high-mean current portfolio while reducing dependence on its weaker worst-window behavior.

3 Results

3.1 Phase2 Blend Evidence

The Phase2 alpha/top-K tuner selected `final_portfolio_a0.45_k44`. Both the active robust score and the older 3/7/13 score selected the same blend. The table below compares the final blend to the two pure source candidates on the same latest 13-window live-style validation run.

Strategy	Robust score	13w mean excess	13w worst	13w hit rate
Phase2 blend, alpha 0.45, top 44	0.558	+0.908%	−2.926%	53.8%
Pure capacity	0.481	+0.768%	−2.458%	55.8%
Pure current	0.300	+0.985%	−3.734%	57.7%

The blend sacrifices some pure-current mean return to reduce reliance on the more parameter-sensitive current portfolio. It also keeps more upside than pure capacity. This is the main empirical reason for selecting the Phase2 blend instead of either pure candidate.

3.2 External Check Against Baseline

The Phase1 portfolio was generated before the May 6–8 prices were known. The same realized window can therefore be used as an external post-submission check of the Phase1 route. It is not used as a static final test for Phase2, because its result was known before Phase2 tuning.

Model	Holdings	Portfolio return	CSI500 return	Excess return
Instructor baseline	50	+6.821%	+4.128%	+2.693%
Phase1 model	31	+8.962%	+4.128%	+4.834%

The Phase1 model exceeded the instructor baseline by +2.141 percentage points of excess return on this external realized check. The exact train/validation/test interpretation of this comparison is documented in the Self-test section.

4 Analysis

What worked. Benchmark-relative labels and features worked better than raw return prediction because the live score is excess return over CSI500. Live-style walk-forward validation was more realistic than a single fixed validation block because each window retrains as if it were a real submission date. Rank-based score blending made the general and short-history XGBoost outputs comparable, while liquidity/risk filters and capped portfolios reduced implementation risk. The Phase2 current/capacity portfolio blend worked because it kept part of the high-mean current candidate while reducing dependence on its weaker worst-window behavior.

What did not work. Standalone IC was too noisy to select portfolios by itself. Short-history-only configurations sometimes looked strong in isolated windows but were unstable. AH-premium data had too little CSI500 coverage and was removed. Pure current had attractive mean excess but larger worst-window losses and higher parameter sensitivity, so Phase2 did not use it alone.

Why the evaluation is still credible. The main risk is overfitting to recent historical windows. This is why the report avoids presenting a repeatedly inspected fixed block as a clean test set. Instead, validation evidence is separated from external realized evidence: validation chooses the model, Phase1 is reported as a realized post-submission check against the instructor baseline, and Phase2 was the true unseen course evaluation for the final submitted portfolio.

5 Self-test

5.1 Train/Validation/Test Protocol

The supervised target is `target_excess_5d`: a stock's future five-trading-day return minus the future five-trading-day CSI500 return. Because the label looks five trading days forward, every training and validation step uses a five-day cutoff or embargo.

Layer	Definition
Training	For each as-of date, train only on rows whose 5-day forward labels are fully known by that date. For example, a 2026-04-30 prediction row can only use labels through 2026-04-23.
Internal validation	Time-ordered validation blocks before the prediction date, with a five-trading-day embargo, are used for XGBoost early stopping and diagnostics.
Model/portfolio validation	Candidate comparison uses live-style walk-forward validation: retrain from scratch at each historical entry date, build a legal portfolio, and score the following 5-day window.
Held-out test / self-test	The Phase1 live window, 2026-05-06 to 2026-05-08, is used as the reported test set against the instructor baseline. It was not known when the Phase1 portfolio was generated.
No reused static test block	After a realized window has been inspected, it is no longer treated as a clean test for later Phase2 tuning. Phase2's true test was the unseen course live window after submission.

This is a train/validation/test split adapted to a sequential portfolio problem. Validation is not a random holdout; it is a set of historical live-style simulations. The test set reported here is the first submitted course window after it became realized. The protocol avoids the main leakage risk in this task: a 5-day forward label whose future return window overlaps the validation or live holding window. After hyperparameters are frozen, the final live model is refit on all labels known at the submission date.

5.2 Concrete Self-test Split

The Phase1 portfolio was generated before the May 6–8 prices were known. This is therefore the report's concrete self-test against the instructor baseline: both portfolios are evaluated on the same realized out-of-sample holding window, and the model pipeline was fixed before that window's labels were known.

The Phase1 model exceeded the instructor baseline by +2.141 percentage points of excess return on this held-out external check.

5.3 Data And Leakage Controls

The data pipeline uses public market data, public index/ETF proxies, Shenwan industry data, valuation data, and announcement-date-aware fundamentals. The important leakage controls are:

- Training labels are cut off by the 5-trading-day horizon, so a prediction row never trains on labels whose return window extends past the as-of date.

Split	Dates / cutoff	Use	Leakage control
Training	As-of 2026-04-30; labels known through 2026-04-23	Train Phase1 model from scratch	No May 6–8 price or label information is available
Validation	Historical pre-2026-04-30 live-style windows with 5-day embargo	Feature/model/portfolio selection	Each simulated entry date trains only on labels known then
Test	2026-05-06 to 2026-05-08	Report held-out performance versus baseline	Scored only after Phase1 submission
Baseline	Rebuilt as-of 2026-04-30	Same test-window comparison	Same price window and benchmark return

Model	Holdings	Portfolio return	CSI500 return	Excess return
Instructor baseline	50	+6.821%	+4.128%	+2.693%
Phase1 model	31	+8.962%	+4.128%	+4.834%

- Fundamentals are joined only after their announcement dates, not by fiscal period alone.
- Feature selection and model tuning use only historical windows available before the simulated entry date in live-style validation.
- The Phase1 data snapshot ended at the April 30 A-share row; Phase2 used the newer May 8 row after those prices were public.

6 Reproducibility And Acknowledgement

Exact reproduction commands and hashes are documented in `README.md`, `docs/PHASE_REPRODUCTION.md`, `docs/DATA_PIPELINE.md`, `docs/MODEL_SELECTION_LINEAGE.md`, and `docs/HYPERPARAMETERS.md`. The baseline comparison commands and outputs are in `experiments/self_test/SELF_TEST_REPORT.md`.

Instructor baseline files `baseline_xgboost.py` and `features.py` are retained unchanged for comparison. LLM assistance was used for coding, documentation, and brainstorming, but the submitted stock weights were produced by the student-trained XGBoost and portfolio-construction pipeline, not by prompting an LLM for stock picks.